Agentic Coding for Economists

Five-hour hands-on workshop

Participant repo: `https://github.com/meleantonio/AC4E_Padova`

Antonio Mele
University of Padova

How We Teach Today

Demo-first rhythm

1. **Short demo ($\leq$8 min):** One tactical workflow, live.
2. **Hands-on sprint:** You produce a file in *your* `examples/starter_article/` copy.
3. **Debrief:** What did the agent do vs what did you sign off?

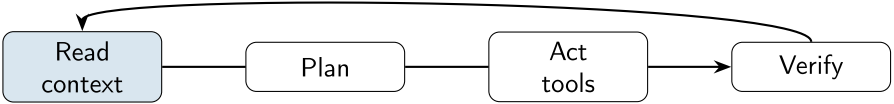
What is Agentic Coding?

Definition

Agentic coding = delegating implementation work to AI agents that **read**, **plan**, **edit**, and **verify**—while you orchestrate scope, context, and quality gates.

- **Agents** operate on your codebase with *tools*: edit files, run terminal commands, search the repo (and optionally the web).
- **You** control what they see (context files, attachments, web resources) and how they behave (the harness: `AGENTS.md`, rules, skills, hooks, ...).
- **Output** must be *verifiable*: tests pass, replication is valid, acceptance criteria achieved.

The Agent Loop



- **Read** — Project brief, `AGENTS.md`, attached files, search results. Bad context $\Rightarrow$ bad code/output.
- **Plan** — Break work into steps (Plan mode, specifications, requirements, tasks, explicit goal).
- **Act** — Edit code, run code/tests, execute terminal commands, update paper write up.
- **Verify** — Re-run scripts, check replication pipeline, etc.

Why Economists Need This

- **Pipeline complexity:** Data $\rightarrow$ cleaning $\rightarrow$ estimation $\rightarrow$ tables $\rightarrow$ paper—dozens of small, automatable steps.
- **Reproducibility pressure:** Journals and replicators expect documented data, portable paths, one-command runs.
- **Multi-artifact projects:** Code, LaTeX, appendix, and replication package must stay consistent.
- **This workshop:** Build a *harness* you can fork onto a research project, not just one-off chat prompts.

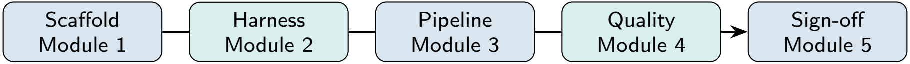
What is a Research Article Harness?

Harness

A **project template** where context files, skills, subagents, hooks, and verification gates work together across the research lifecycle, from idea to replication.

- **Not** a single magic prompt.
- **Is** repeatable infrastructure: the agent always knows the repo map, boundaries, and how to run checks.
- **Can** be adapted to other research projects.

Harness Lifecycle (Workshop Spine)



- **Scaffold** — Create repo, project idea/brief, `AGENTS.md`, privacy boundaries, three tasks.
- **Harness** — Create skills, subagents, hooks, MCPs; run one long goal with stop conditions.
- **Pipeline** — Update idea/brief, write and maintain code and paper.
- **Quality** — Referee report, replication check
- **Sign-off** — Checklist + 7-day adoption plan for your own research.

Demo: Coding Agent Interface

A typical coding agent UI:

- **Prompt/Chat:** Type a goal
  (e.g., "Clean `data.csv` and compute summary stats.")
- **Context** loaded: repo map, `AGENTS.md`, `templates/project_brief.md`.
- **Artifacts:** View/edit code, instructions, logs, or generated files.
- **Tools:** Run code, preview plots, test outputs (e.g., buttons for "Run All", "Test", "View PDF").

Project Context: `AGENTS.md`

What it is

The "front door" for any agent: a human-readable brief at the repo root that loads every session.

Typically includes:

- **Research question** and stage (seminar draft, JMP, etc.)
- **Repository map** — which folders agents may read or write
- **Verification commands** — `pytest`, `src/estimate_did.py`, `latexmk`
- **Do-not rules** — e.g. never edit `data/raw/` without permission

Claude lane: may also use `CLAUDE.md`; keep `AGENTS.md` for collaborators.

Example: AGENTS.md (HANK Macro)

Same pattern for applied micro, structural, or macro repos.

```
# AGENTS.md - HANK macro (illustrative)

## Project
HANK: monetary policy with heterogeneous households; seminar draft.

## Repo map
Read: src/, models/, data/processed/, docs/
Write: src/, models/, docs/drafts/, output/
Do NOT: data/raw/, output/confidential/, secrets/

## Verify
pytest src/ | python models/run_HANK.py | latexmk -pdf docs/paper.
   tex

## Rules
No raw-data edits without permission; no API keys or microdata in
   context.
Tests must pass; document model changes in docs/; use a branch for
   review.
```

Exercise: Create Your Own `AGENTS.md`

Exercise: Draft an `AGENTS.md` for your own project.

Include the following sections

- **Project:** Briefly state your project's research topic and stage.
- **Repo map:** *Which folders should your agent read or write?* What should be off-limits?
- **Verify:** Specify at least one command to check code or outputs (e.g., `pytest`, `make`).
- **Rules:** Any do-not rules, data boundaries, or requirements for contributors/agents?

Tip: Imagine onboarding a new research assistant (human or AI)—what context and boundaries do they need on their first day?

(Start from `templates/AGENTS_economics.md` or the HANK macro example above.)

Research Spec: Brief and Design Memo

Project Brief:

`templates/project_brief.md`

- **What:** Scope, audience, in/out of scope
- **Why:** Stops agents from "helpfully" expanding the paper
- **When:** At the beginning of the project, Module 1 — personalize or keep demo

Example:

`examples/starter_article/docs/project_brief.md`

Design Memo:

`templates/research_design_memo.md`

- **What:** Hypotheses, data, identification, risks/threats
- **Why:** Plan before code; hub for SDD-lite
- **When:** Beginning of the project, but maintained throughout, Module 3 — identification section is human-owned

Example:

`examples/starter_article/docs/research_design_memo.m`

Spec-Driven Development:

`examples/starter_article/spec/intent.md` — optional; for ambitious projects (intent → requirements → design → tasks).

Exercise: Create Your Own Project Brief and Design Memo

Exercise: Draft a project brief and design memo for your own project. Start from `templates/project_brief.md` and `templates/research_design_memo.md`.

Project Brief

- **What:** Scope, idea,audience, in/out of scope

Design Memo

- **What:** Hypotheses, data, identification, risks/threats

Privacy Boundary: `.cursorignore` or `AGENTS.md`

What it does: Controls what the agent *indexes* and may send as context.

- **Privacy:** API keys, microdata, confidential drafts
- **Performance:** Large CSVs, PDFs, `.venv`

Different from `.gitignore`: `.gitignore` = version control; `.cursorignore` = AI context.

```
data/raw/confidential/
.env
*.dta
*_confidential*
```

What Are Agent Skills?

Skills extend the agent with **reusable workflows** stored as `SKILL.md` files.

- **Trigger:** The skill's `description` in YAML frontmatter matches your request.
- **vs subagents:** Skills augment the *main* agent—no separate context window.
- **vs rules:** Rules are always-on constraints; skills are procedures you invoke.

Exercise: Create Your Own Agent Skills

Exercise: Create an agent skill for your own project.

Skill

- **What:** Description, trigger, steps

What Are Subagents?

Subagents are specialized assistants with their own **context window** and prompt.

- **Why:** Context isolation—a reviewer should not share the same "implementation bias" as the coder.
- **Format:** `examples/starter_article/.cursor/agents/*.md` or `examples/claude/.claude/agents/*.md` with YAML frontmatter + role prompt.
- **Invoke:** "Use identification-reviewer on the design memo only" or tool-specific `/agents`.

Exercise: Create Your Own Subagent

Exercise: Create a subagent for your own project.

Subagent

- **What:** Description, trigger, steps

What Are Hooks?

Hooks run **deterministic** scripts at lifecycle events (session start, before/after tool use).

- **Why:** LLMs forget; hooks enforce reminders (privacy, no `rm -rf`, run tests after edits).
- **Where:** `examples/hooks/.codex/`, Claude `settings.json`, `examples/starter_article/.cursor/hooks.json`.
- **Trust:** Codex `/hooks`; Claude settings UI—you approve what runs.

Economics example: After editing `examples/starter_article/src/`, hook reminds: "Re-run `pytest` and replication-checker before claiming results."

Long-Running Goals and Checkpoints

Problem: One chat cannot hold a whole pipeline; context drifts.

Solution: Explicit goals with **allowed paths**, **forbidden paths**, and **stop conditions**.

Tool	Mechanism
Codex	`/goal` or goal UI; `codex resume`
Claude Code	Checkpoints; `/goal`; `claude -continue` (CLI only)
Cursor	Background agents; Agent checkpoints

Exercise: Write a Long-Running Goal

Exercise: Draft a long-running goal for your project using the following template.

Long-Running Goal Template

- **Goal:** (Describe your end goal or desired outcome clearly and concretely.)
- **Allowed paths:** (List productive actions the agent can take to achieve the goal.)
- **Forbidden paths:** (List actions or areas that are off-limits; e.g., avoid modifying published data, do not email external parties, etc.)
- **Stop conditions:** (Define clear stopping criteria, e.g., when the goal is achieved, a report is generated, all code passes tests, etc.)

Tip: Be explicit about boundaries and what success looks like. This helps both you and your agent stay aligned over time. Template: `templates/long_running_goal_prompt.md`

MCP Servers

MCP = Model Context Protocol — a standard to connect agents to **external tools**.

- **What:** Cursor (or other hosts) invoke MCP servers; the agent sees structured tools (e.g. FRED API).
- **Config:** `examples/starter_article/.cursor/mcp.json.example` → `.cursor/mcp.json`; API keys via environment variables, never in git.
- **When it helps:** Recurring macro series pulls, scoped filesystem access.
- **When to skip:** One-off downloads; workshop runs fine without FRED.

Verify MCP setup in your Cursor version. Docs: https://cursor.com/docs/context/mcp

Skills vs Subagents vs Main Agent

Use	When
Skill	Single-purpose procedure (replication-check, referee pass, SDD plan)
Subagent	Isolated review role; parallel second opinion on memo or PR
Main agent	Orchestration, merging edits, deciding scope

Invoke explicitly: "Run replication-checker on repo root" beats "check if this replicates."

Swarms: GitHub as Control Plane

A **swarm** coordinates several agents toward one project outcome — not one giant prompt.

- **Issues** = units of work with acceptance criteria and allowed files.
- **Labels:** `parallel`, `sequential`, `blocked-by:#N`, `ready-for-review`.
- **Rule:** independent tasks → parallel agents; dependent tasks → sequential launches after prior PR merges.
- **Planner** opens issues; implementers work branches; reviewers stay read-only.

Workshop plan: `orchestration/card_krueger_swarm.md`; issue template: `orchestration/cloud_agent_issue_template.md`

Example: Card-Krueger Swarm Streams

Stream	Label	Depends on	Reviewer
Data map audit	sequential start	none	`data-reviewer`
Cleaning and validation	sequential	data map	`replication-verifier`
Baseline estimate	sequential	cleaning	`replication-verifier`
Source note	parallel	source URLs only	`identification-reviewer`
Teaching write-up	sequential	baseline	`pr-reviewer`

- Record handoffs in `notes/orchestration_log_template.md`.
- Merge only after review; keep `main` stable.
- Neither swarms nor loops replace human merge approval.

Loop on Verification

Three phases, repeated until exit or escalation:

1. **Implement** — narrowly scoped change against acceptance criteria.
2. **Evaluate** — `loop-verifier` subagent or replication-checker skill.
3. **Revise** — address only listed blockers; record iteration in the orchestration log.

Verdicts: GREEN → human review; YELLOW → loop again; RED or 3 iterations → stop and open a new issue. Protocol: `orchestration/verification_loop.md`; skill: `examples/starter_article/.cursor/skills/loop-on-verification/`; evaluator: `examples/starter_article/.cursor/agents/loop-verifier.md`

Loop vs Swarm

Swarm

- Many issues in parallel waves
- GitHub labels and dependencies
- Merge after subagent review
- Example: `orchestration/card_krueger_swarm.md`

Verification loop

- One issue, one agent stream
- Sequential implement–evaluate cycles
- Exit when criteria are green
- Skill: `loop-on-verification`

Both require acceptance criteria first. Neither replaces human merge approval.

Goal Files and the `/goal` Command

A **goal** is persistent task state the agent retrieves across sessions — the local equivalent of a GitHub issue.

Field	**Write verifiable criteria**
name	one-line task identifier
description	economic output when done
approach	files to edit, method to use
acceptance_criteria	checkbox list; each item checkable by command
constraints	raw data untouched; synthetic caveat required

Worked examples: `orchestration/goals/`; template: `templates/long_running_goal_prompt.md`.

Attach via Claude Code `/goal`, Codex Goals UI, or Cursor Cloud Agent task description.

Harness Components (Summary)

Component	What it does (Cursor paths)
Project context	Always-on: AGENTS.md, templates/AGENTS_economics.md, examples/starter_article/.cursor/rules/*.mdc
Research spec	Plan before code: templates/project_brief.md, templates/research_design_memo.md, examples/starter_article/spec/intent.md
Skills	Procedures: examples/starter_article/.cursor/skills/referee-checklist/, etc.
Subagents	Isolated reviewers: examples/starter_article/.cursor/agents/
Hooks	Deterministic guardrails: examples/hooks/.codex/, examples/starter_article/.cursor/hooks.json
MCP	Optional external APIs: examples/starter_article/.cursor/mcp.json.example
Gates	Evidence: examples/starter_article/replication/README.md, templates/referee_report.md, templates/review_log.md
Orchestration log	Lightweight record: what you delegated vs reviewed (notes/orchestration_log_template.md)

Autonomous Agents: OpenClaw, Hermes, Eve

Agent	What to notice	Workshop stance
OpenClaw	Local-first assistant; channels, tools, skills, sandboxing, multi-agent routing	Good for personal-agent and channel-safety discussion; optional reading
Hermes	Nous Research agent focused on long-running context and user-specific capability growth	Useful for memory, persistence, and ownership of research context
Eve	Vercel framework for building agents with instructions, skills, tools, sandboxes, channels, subagents, schedules	Use only after checking current docs and access boundaries

Rule: More autonomy means more need for logs, review gates, and reproducibility evidence.

Autonomous-Agent Risk Card

Fill one risk card before letting any autonomous agent touch a research repo:

- **Read boundary:** what files, channels, or data can it inspect?
- **Write boundary:** what can it edit, trigger, schedule, or publish?
- **Memory:** what state persists across sessions, and who controls it?
- **Execution:** local, cloud, sandbox, or production system?
- **Review gate:** what human approval is required before outputs enter the repo?
- **Evidence:** what logs, diffs, commands, or replication outputs prove what happened?

Artifact: `agent-harness/autonomous_agent_risk_card.md` (see `https://github.com/meleantonio/AC4E_Pavia`)

Risk Card Decision

Approve only if you can answer:

- **Read**: what can it inspect?
- **Write**: what can it edit, trigger, publish, or schedule?
- **Memory**: what persists and who controls it?
- **Execution**: local, sandbox, cloud, production, or mixed?
- **Evidence**: which logs, diffs, transcripts, or outputs prove what happened?

Decision labels

- Safe for read-only use.
- Safe for branch-isolated write use.
- Not safe for this workshop repo.
- Needs more documentation before use.

Rule: do not connect private data unless every boundary and approval gate is acceptable.

Map New Agents Back to the Harness

Autonomous-agent feature	**Course harness equivalent**
Markdown instructions	`AGENTS.md`, `CLAUDE.md`, Cursor rules
Skills / tools	`SKILL.md`, MCP tools, validation scripts
Postconditions	Hooks, CI, verification commands
Task memory	Goal files, GitHub issues, orchestration log
Channels / schedules	GitHub issues, CI, scheduled checks
Subagents / workers	data-reviewer, literature-reviewer, loop-verifier
Sandbox	Branch, worktree, remote runner, protected main
Logs	PR description, orchestration log, replication README

Teaching point: The architecture transfers; the UI changes.